

Engineering Manager — Take-Home Submission

Parts A, B and C. Assumptions are in `NOTES.md`. Technical terms are explained where they appear, so this reads without knowing the codebase.

Part	Page	Limit	Mine
A — 90-day plan	2–4	3 pages	3 pages
B — the review	5	1 page	1 page
B — the conversation	6	½ page	290 words
C1 and C2	7	300 words each	243 and 257

Part A — My 90-day plan

1. What is actually wrong

This team is good at fixing things and bad at remembering them.

PR #482 goes live tomorrow. I review it in Part B. It brings back four problems we already had in the last 90 days:

- It can pay the same person twice (incident #1).
- It stores money in a format that rounds the number (incident #4).
- It runs a huge job inside one web request, which will block the database (incident #2).
- It pays every brand's members when it should pay one brand's — same shape as #3, wrong settings in the wrong region.

A staff engineer read it and approved it.

So this is not about skill. Nobody writes down what went wrong. There is no incident process and the region-3 review never happened, so the lesson stays with whoever fixed it.

The money rules are advice, not rules. “Store money exactly. Always filter by brand.” That lives in a README and in S.'s head. Nothing checks it automatically, so one person reading a code change is all that stands between a mistake and real money. That is why 3 of the 5 smaller incidents came from payment code that *had been reviewed*.

Two people carry too much. S. is the only one who really knows the wallet code. T. is the only one who knows the data pipeline, and he is on call for it too — #10 and #11 happened only because he was on leave. S.'s reviews are hard on juniors, so juniors avoid the wallet code, which makes S. even more necessary. It feeds itself.

QA comes last, so bad news comes late. Two people test at the end. Problems appear late, the release slips, the next release gets bigger, and a bigger release breaks more. That is why releases went from weekly to every two weeks. Late is also expensive: a bug caught while the developer still has the code open costs an hour, and in production the same bug costs a support queue, a rushed fix, and sometimes real money — incident #1 was clawed back by hand.

Nothing is measured, so “region 4 in 10 weeks” is a promise nobody can check. Region 3 took 4 months against a 6-week plan and nobody asked why.

The CEO's two goals are one goal. Region 4 means a new market, new payment providers and different rules — exactly the payment work that causes our money incidents. So the launch and “no money incidents” are not two jobs fighting each other. Making payment changes safe *is* how I launch on time.

2. What I will do, and what I will not do

Days 1–14: learn, and put up the cheap guards. One-to-one with all 22 people. Read every incident ticket. Then ship the guards nobody needs to agree to:

- Automatic checks that reject money stored in a rounding format, and reject unsafe money maths.
- Database rules that make a double payment impossible, and stop a balance going below zero. A rule only works if the key it checks is stable, so the key comes from the event — member plus week — never from the clock, and each guard ships with a test that runs the job twice and expects one payment. PR #482 shows why: its week value means “seven days before whenever you ran it”, so two runs the same day slip past a unique index.

A few days of work, and it catches #1, #4 and PR #482. Plus a one-page incident process: who declares, who does what, write the timeline, blame-free review within 5 working days, one owner per action.

Days 15–45: use region 4 to fix things. Build one standard shape for payment providers, so adding one means settings and two small functions instead of touching the wallet core. The same work makes the launch faster and safer. Put each region's settings in code, review the differences before launch, and dry-run against the providers' test systems. That is aimed at #3.

Spread knowledge now, not later. S. works with two mid-level engineers on the wallet code. T.'s on-call becomes two people with a written runbook. And testing moves earlier: QA writes the test cases, developers run them before opening the PR, and QA keeps the difficult edge cases. Their skill is knowing what breaks, not clicking through steps.

Days 46–90: launch, then prove it. Region 4 at week 10, with a go/no-go check first. Publish the review afterwards, the one region 3 never got. Then 60 days of numbers, so we see a trend.

What I will not do this quarter. Saying yes to everything is how region 3 became 4 months.

- **No team reorganisation.** Changing teams during a launch is two risks for one benefit.
- **No rebuild of the one-deployment-per-region setup.** The real cause behind #3, but a year of work. Settings in code buys most of the safety cheaply.
- **No new infrastructure** beyond moving big jobs out of web requests, and **no bug-backlog crusade** — the +30% is a symptom, so I stop the inflow and clean the list once.
- **CRM & Data gets slower.** I only fix the “T. is the only one who knows” problem, and I say out loud to T. and to the business that internal reports will sometimes be late.
- **No formal warning for S., and no promise to return to weekly releases.** Releases get faster when tests are automatic, not because I set a date.

3. Process changes

Four only. Each comes from an incident that really happened. If I cannot name the incident, it does not go in. That rule is also my answer to L. in C2.

1. **Rules for money code.** Any change to wallet, ledger, payout or provider code needs a key built from the event and not from the clock, a test that runs it twice and a test that runs two copies at once against a real database, and a second reviewer from another squad. An AI reviewer runs first on every money PR, against our written rules: exact money types, brand filter, key from the event, one transaction, no unbounded loop inside a request. It comments and never approves — accountability stays with a person. It is also where post-mortem lessons land, so a rule we write gets applied to every PR instead of sitting in a document. *Fixes:* 3 of 5 incidents came from reviewed payment code. *Works when:* payment code stops appearing as a cause, and every money change has that test.
2. **A blame-free incident process.** Written review in 5 days, one named owner per action. *Fixes:* we do not turn incidents into fixes, and repair time is getting worse with nothing measuring it. *Works when:* the same cause stops coming back and repair time falls.
3. **Move testing earlier, instead of QA signing off at the end.** QA writes the test cases, the developer runs them before the PR is open, and QA takes only the hard edge cases. Those edge cases are the safeguard: without them this turns into developers marking their own homework. *Fixes:* QA is a queue, and a bug found late costs far more than the same bug found during development. *Works when:* most bugs are found before merge instead of in QA or production, QA hours per release fall, and releases get faster.

4. **A launch checklist, plus reviewing settings differences between regions.** From #3. *Fixes:* launches break on settings, not code. *Works when:* region 4 launch day has no major incident.

4. What I will measure

- **Money incidents, and money at risk each time.** What the board cares about. I also count **near misses caught in review**, because a zero from people not reporting means nothing. PR #482 is a near miss and I want it counted.
- **How often the same cause comes back.** The direct test of whether we learn. Given what I found, this is the number I would want to be judged on.
- **How often payment changes break, and how long we take to fix them.** Payment code only. A company-wide average hides the one place where a mistake is serious.
- **Time from merge to live.** Measures our release process, not how fast people type.
- **How many people really know each of our five most important systems.** Counted as: wrote code in it, reviewed it, and was on call for it in the last 90 days. Makes the S. and T. risk visible.

Not measured: story points, velocity, pull requests per person. Those punish the pairing and writing-down this plan depends on.

5. People

- **S.** — my biggest win. He is not a blocker to remove, he is an architect being used as a gate. I change his job: instead of reviewing everything he writes the rules — the money checklist, the automatic checks, the database rules — so a machine enforces his standards and juniors stop crying. The AI pass also takes the mechanical nitpicks off him, so his comments are about design. I coach him on review tone, using PR #482 as the example. Not optional: two mid-level engineers able to work in the wallet code by day 90. If his tone has not changed in 60 days, that becomes a performance conversation.
- **D.** — support, not blame. He wrote what this team taught him was acceptable, and a staff engineer agreed. He builds the double-payment guard with S., because you learn a rule by building it.
- **T.** — protect him first. On-call becomes two people, another backend engineer works alongside him, the runbook gets written. I tell him directly that his squad's backlog is not the priority. If he learns that by failing, I lose him.
- **L.** — make the mentoring he already does official, and say so publicly. Then make him co-owner of the release checks, not just someone who receives them. He is also the most likely person to tell me in public that I am wrong, which is exactly C2. I would rather he tested this plan before I announced it.
- **The three Payments juniors** — put them on the guard work. They learn money code by building the safety checks, and it is the cheapest way out of the "only S. knows" problem.
- **The two QA** — their job changes from running tests to designing them. They write the test cases, developers run them, and QA keeps the hard edge cases and the automation. I will not hire more manual testers; that makes the queue longer, not shorter.
- **The two open backend roles, empty for 3 months.** That means the job description or our interviews are wrong, and finding out is week-1 work for me. I change them to one senior backend engineer with real payments experience plus one platform engineer, not two mid-level people: Payments needs a second person who can carry the wallet code, and mid-level hires would add review work for S. Being honest, neither helps the 10-week launch, because learning this system takes longer. So I plan region 4 as if they do not exist.

Part B — PR #482

The review, as I would post it

What I would type into the PR, in this order.

Top comment. Thanks D. The layout is right and the description made this quick to review.

I am blocking, and it is not about your ability. Four problems below already hit us in the last 90 days: paying twice (#1), money in a rounding format (#4), a big job blocking the database (#2), and a query that forgets which brand it is for, same shape as #3. A staff engineer approved it too. That they came back and still passed review is a tooling problem, not a you problem. I am opening a ticket for that and would like you to own part of it. Nothing needs a redesign: a transaction, `dec()`, a unique index, a `WHERE account_id`.

cashbackService.ts:78 — blocking. `Member.findAll()` has no filter. The endpoint takes `accountId` and says the run is per brand, but nothing uses it. So one brand's run pays every member of all ~30 brands, from that brand's budget. `Bet.findAll` below filters by member only, so it misses it too.

migration:22, cashbackPayout.ts:53, :92, :106 — blocking. The column is `FLOAT` and the code uses `parseFloat` on the amounts. `FLOAT` cannot hold exact decimals: it rounds, and the error grows as you add up a week of bets. That is #4, except that was a report and this puts money in wallets. Use `DECIMAL(36,18)`, keep the field a string, do the maths with `dec()`, and pick a rounding rule for `netLoss × 0.05` and write it down. And can `payout` be empty for an unfinished bet? Then `parseFloat(null)` gives `NaN`, “not a number”. `NaN <= 0` is false so we do not skip, and we save `String(NaN)`, which Postgres accepts in a numeric column. That balance cannot be repaired.

:76, :109 — blocking. Nothing stops this running twice: no unique index on `(member_id, week_start)`, no check for a payout that exists already. Worse, `weekStart` is `Date.now() - 7 days`, which is “seven days before whenever you called it”, not “the start of the week”, so two runs on the same day give different values and even a unique index would not stop them. Needs a real week id like `2026-W28`, a unique index on it, and a ledger key from member plus week. Ops runs this from cron, and cron retries.

:101–124 — blocking. Balance, payout row and ledger row are three separate writes with no transaction, so they cannot all succeed or all fail together, and the `catch` just prints and carries on. If the second one fails, the member has money with no ledger row explaining it, and the balance stops adding up from the ledger. One transaction per member, and record failures somewhere we can query. Also `wallet.balance = ...; save()` reads the balance, adds to it, then writes the whole row back with no lock — so a deposit landing mid-run gets overwritten and disappears. That is the player's money, not ours.

:78–125, admin.ts — blocking. 40,000 members × about 5 queries in one HTTP request. The app keeps only a few open database connections, so this takes them all and other requests start failing. That is #2. It also times out, ops retry, and since nothing stops a second run the retry pays everyone again. Return a run id and work in small batches. Also `/admin` has no login check: anyone who can reach the service can move real money.

test/cashback.test.ts — blocking. Every database call is faked, so this passes with everything above still broken. It checks `save()` was called, never that the amount is 3.00. Money tests need a real database: run twice at once and check we pay once, fail halfway and check nothing is half-written, run one brand and check another is untouched.

The conversation

D. first, before I post. A short message first, so seven blocking comments from the new manager do not hit him cold. Then the review in writing, so his squad sees the reasoning. He wrote what this team told him was acceptable and a staff engineer agreed, so he gets the fix to own: he builds the double-payment guard with S., because you learn a rule by building it.

S. in private, not on the PR. Arguing with a staff engineer's approval in public costs more than this PR is worth, and makes him defensive right when I need him teaching. The real issue is not that he missed things. It is "LGTM, nice and simple" on code that moves money. His advice to D. — keep it simple, no queue — was right about complexity and wrong about how much damage it could do. His approval was our only safety check, and a check that fails is a reason to automate his standards, not to question his attention. He writes those checks, so the next PR like this fails before a human reads it.

What I change so this does not land on my desk next time. Me happening to read a PR on day 9 is luck, not a process. So: the money rules become a checklist in the PR template; an AI reviewer checks it on every money PR; automatic checks reject `FL0AT` and raw `parseFloat` on money and flag queries missing the brand filter; money code needs two reviewers, one from another squad; and database rules reject the dangerous version even if review misses it. Every problem I found maps to one automatic check. My job is to build the safety net, not to be it.

Part C

C1 — Saturday 01:40, wrong balances in region 2

Assumption: withdrawals are the only way money leaves for good.

Minute 0–10, set up. I declare an incident. S. leads the technical side, because I do not know this code and saying so is faster than pretending. I take comms, one Payments engineer keeps a timeline, and T. stays on leave.

Minute 10, stop the bleeding. Turn off the suspect credit path and stop withdrawals in region 2. Angry players and a busy support queue beat money leaving for good while we still do not know why.

Minute 15, reply to the CEO. A number, or an honest “we do not know yet”. Never “it is fine”. *“Maybe. Withdrawals in region 2 are stopped, so nothing is leaving for good. It looks like X across N accounts. Next update 03:00.”* Then I update at that time, news or not.

Hour 1–3, get the real number. Compare every balance against the ledger. That number decides everything else, including whether we have to tell the regulator. Compliance owns that call by morning.

Hour 3–12, fix forward. Find the cause, fix it, then repair the balances with a reviewed script that writes proper ledger rows. No hand-editing the database, and no manual clawback like #1. Here the paper trail is the point.

Before anyone sleeps. Tell the brands once the numbers are final. Hand over so S. can rest. Book the blame-free review.

C2 — Week 6, L. calls the plan process theater

In the room, first 60 seconds. I do not defend the plan and I do not shut him down. The room is not asking about a checklist. It is asking whether disagreeing with me is safe, and it learns that from my face, not from my plan.

So: *“That is fair, and I would rather hear it now than in a retro in six weeks. Which part is slowing you down, all of it or specific items? If it is hurting region 4, that is my problem to fix.”* Then he talks and I take notes.

I close with a promise, not a decision: we measure what the checklist costs per release and drop it by week 8 if it is not worth it. I do not settle it there, put it to a vote, or let “just ignore it” stand.

After. A one-to-one inside 24 hours, and I listen first. Either he is right and part of it is pointless, or he is shielding his team from work that has not paid off yet. Either way he knows something I do not.

The real problem is mine. I announced a process instead of building it with the leads, so the most respected lead had no stake in it. So L. co-owns the release checks. If he cannot defend them to his own team, they are wrong.

Then I close the loop in the same meeting. Dissent that visibly changed something is the cheapest proof it is welcome.

AI use, as the brief requires: I used Claude Code to read the handover pack and the PR #482 diff, to cross-check that diff against the incident table, and to cut all three parts down to the limits. Two of the technical findings came from it and I checked both against the diff myself. The judgements and recommendations are mine.